# ARCHITECTURE DECISION RECORDS

**ADR - 001**: Domain Model como patrón de lógica de negocio

Estado: Aceptada

Contexto

RutaNorte Logistics transporta mercancía entre Monterrey, Saltillo, Reynosa, San Luis Potosí y Querétaro. Necesita digitalizar y optimizar el seguimiento de una orden/pedido/servicio de principio a fin. En el que los operadores dejen de comunicarse por medios informales como WhatsApp, tener registro de las fases o
estados en tiempo real y obtener una organización en cuanto a las evidencias presentadas.

Alternativas

- Transaction Script: organiza la lógica de un negocio en diversos scripts, en procedimientos individuales, donde cada uno de estos maneja un solo flujo completo, o sea un solo procedimiento.
- Table Module: funciona en base a tablas. Organiza la lógica con una clase por tabla de la base de datos, y una única instancia de una clase contiene los distintos procedimientos que actuarán sobre los datos.
- Domain Model (seleccionada): tiene un enfoque donde la lógica de negocio se organiza dentro de una red estructurada de objetos que contienen tanto datos como comportamiento, y este interactúa activamente aplicando las reglas del negocio.

Decisión

RutaNorte Logistics funciona a base de un ciclo de vida de 8 estados aproximadamente, donde pueden haber cambios y excepciones documentadas y reglas de negocio que dependen del estado del objeto. Es por ello que el patrón más adecuado para RutaNorte Logistics es el patrón Domain Model.

Consecuencias

Beneficios: un cambio en una regla de negocio (por ejemplo, el porcentaje de penalización por cancelación) se modifica en un solo lugar y también se cambia automáticamente en todos los casos de uso que lo invocan

Costos: este modelo es más complejo que los otros dos patrones, este modelo puede llegar a ser mucho más mantenible. Sin embargo, este requiere un proceso inicial más complejo, al identificar los Aggregates, las Entidades, los Value Objects, etc.

Riesgos: se necesita disciplina del equipo para no filtrar lógica de negocio hacia los Application Services (lo que rompería el principio de Service Layer) ni hacia la capa de infraestructura (lo que acoplaría el dominio a detalles de base de datos).

Trabajo posterior: implica diseñar interfaces de Repository y Unit of Work para separar la persistencia del dominio, y mantener cada Domain Service con una sola responsabilidad. También implica que cualquier nueva regla de negocio futura deberá evaluarse primero para decidir en qué objeto de dominio vive, en lugar de añadirse directamente a un Application Service.

Trazabilidad

N-06, N-08, N-09, N-10, N-12